

From LLM translation to validated NPU kernels

Agentic feedback improves completion; hardware testing is the next milestone.

01 / TRANSLATION

70% → 95%

Historical task completion

One-shot → agentic
14/20 → 19/20 cases

02 / CORRECTNESS

22/22

Current offline checks pass

Deterministic candidates
264 host input cases

03 / HARDWARE

0 runs

NPU execution blocked

Latency and power
are unmeasured

Different evidence sets: the 22-case evaluator suite is not a rerun of the historical LLM comparison.

One-shot translation is not reliable enough

10 kernels × 2 NPU targets; one model call and one compile attempt per task.

KERNEL	AMD	Intel
CUDA · Reduce sum	PASS	PASS
CUDA · Tiled matrix multiply	PASS	PASS
CUDA · Tiled transpose	PASS	PASS
CUDA · Vector add	FAIL	PASS
HIP · Gamma correction	PASS	PASS
HIP · Image convolution	PASS	FAIL
HIP · Moving average	FAIL	FAIL
Triton · Fused attention	FAIL	PASS
Triton · Fused softmax	PASS	PASS
Triton · Layer normalization	FAIL	PASS

14/20

tasks completed

70% overall completion

6 failures

across both targets

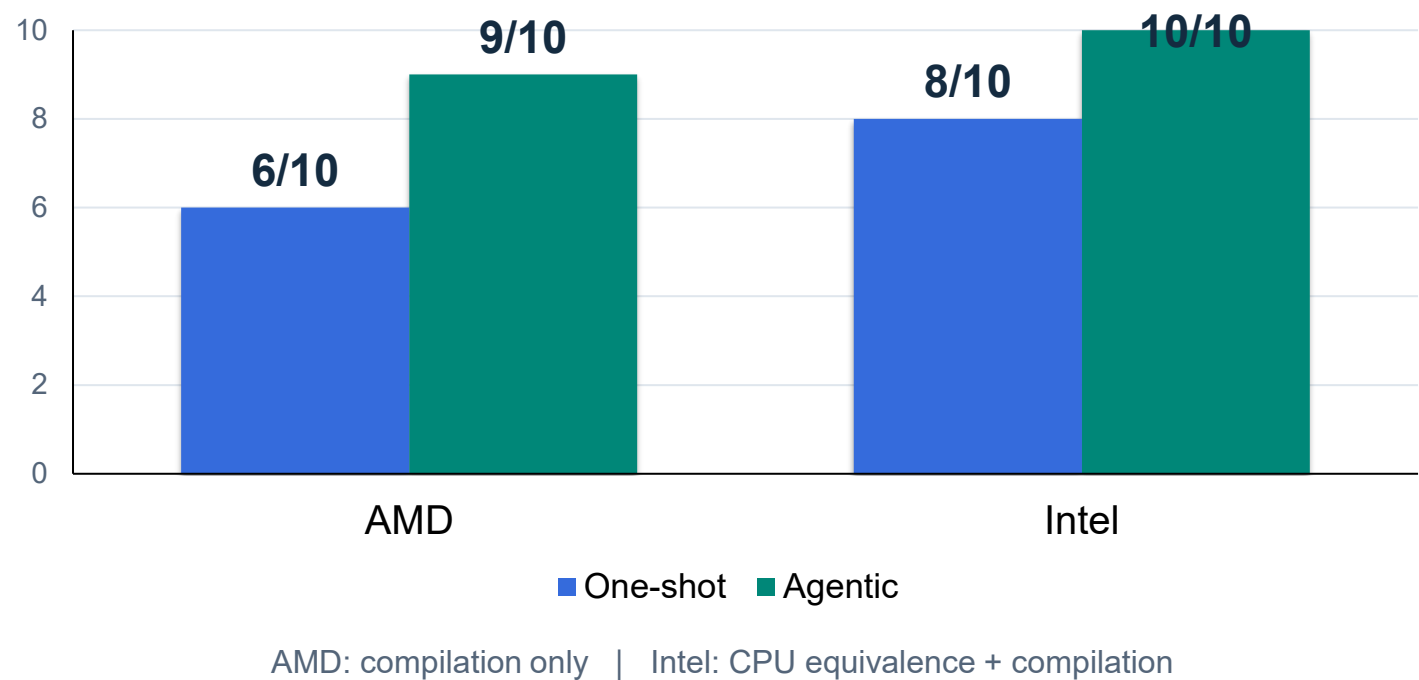
AMD: 4 | Intel: 2

This is one evaluated provider configuration, not a survey of all LLMs.

PASS criteria: AMD = compilation only; Intel = historical CPU equivalence + NPU compilation.

Agentic feedback recovers 5 of 6 failures

The framework analyzes, generates, compiles and repairs candidates using tool feedback.



+25 pp

completion improvement

70% → 95% | 14/20 → 19/20

1 failure remains

AMD fused attention failed the intermediate-representation check.

SEARCH COST

Model calls: 20 → 127

Compile attempts: 20 → 216

This shows a benefit from the full agentic workflow; it is not an equal-compute comparison or a 95% device-correctness result.

The framework now checks correctness explicitly

Independent references, host execution and AMD dataflow checks form the offline acceptance gate.



22/22

offline cases pass

11 AMD + 11 Intel candidates

264

host input cases checked

12 inputs × 22 candidates

396

AMD case / schedule checks

132 inputs × 3 schedules

Current validation gate	AMD	Intel
Host output + target compilation	11/11	11/11
FIFO / DMA dataflow simulation	11/11	Not implemented

Faults caught: wrong arithmetic → mismatch | bad DMA stride → bounds error | missing FIFO release → deadlock

These deterministic cases validate the evaluator; the historical LLM experiment has not been rerun under this stricter policy.

AMD and Intel validate different representations

Both paths can check supported computations and build target artifacts inside containers.

AMD

XDNA2 / npu2 / AIE2P

EXECUTES ON HOST

Candidate C++ through audited host support; outputs checked against a reference.

MODELS DATA MOVEMENT

Bounded MLIR FIFO / DMA model checks transfers, bounds and deadlocks.

COMPILES FOR NPU

MLIR-AIE / Peano produce target artifacts, including xclbin and instructions.

Intel

OpenVINO / NPU 4000

EXECUTES ON CPU

Candidate OpenVINO graph runs on the CPU backend; outputs checked against a reference.

CHECKS GRAPH SUPPORT

Offline NPU compilation tests whether the pinned compiler accepts the graph.

COMPILES FOR NPU

The compiler exports an NPU blob. CPU execution does not execute that blob.

Shared gap: 0 exact target-binary executions; all 22 corpus cases report no validated target executor.

A correct kernel still needs a hardware benchmark

Edge workloads need acceptable latency and energy use as well as correct outputs.

CORRECTNESS

Offline checked

Does it compute the right result?

Source / graph evidence exists.
Exact binary still needs NPU testing.

PERFORMANCE

Unmeasured

Does it meet the response budget?

Measure NPU p50 / p95 latency and completed work per second.

POWER + ENERGY

Unmeasured

Does it fit the device's energy budget?

Measure average watts and millijoules per completed inference.

Container checks cannot establish NPU speed, power or energy per inference.

Measure on AMD and Intel hardware; state whether power covers the NPU, the board or the whole system.

Close the loop on real AMD and Intel hardware

The next result should connect translation success, device correctness, latency and energy.

01 Rerun the translation comparison

Use the upgraded correctness policy for both methods.

Pin the model and compare matched tasks and budgets.

02 Execute the exact target artifacts

Connect each NPU, its drivers and a validated device runner.

Compare device outputs with the same independent references.

03 Measure and optimize for the edge

Collect latency, throughput, watts and energy per inference.

Compare only candidates that pass device correctness checks.

Deliverable: a correctness-filtered latency and energy comparison for both NPU targets.

What the numbers prove — and what remains open

All charts and counts are backed by the saved project reports; source hashes are in evidence.json.

E1 / Historical translation experiment

40 recorded cases: 20 per method

10 kernels × AMD and Intel

Revision: 74f8ae752b23

Codex CLI 0.144.5; model: CLI default

AMD compile-only; Intel CPU + compile

Unequal search budgets

Supports workflow completion, not device correctness.

E2 / Current evaluator validation

22 deterministic candidate-target cases

12 inputs per candidate; 3 AMD schedules

Base: d3efa60a08e5 + working-tree upgrade

0 provider calls; 0 target binary executions

Independent references and negative checks

Not a rerun of the historical LLM experiment

Supports scoped offline checks, not exhaustive proof.

PUBLIC CONTEXT — CLICKABLE SOURCES

[P1 AMD dataflow pre-simulation](#)

[P3 MLPerf Tiny: quality, latency, energy](#)

[P2 Intel OpenVINO NPU device](#)

[P4 MLPerf Edge: power measurement](#)